

Shell

PENG

October 3, 2026

Contents

1	Introduction	1
2	Shell Syntax	1
2.1	Quoting	1
2.2	Pipelines	2
2.3	Lists of Commands	2
2.4	Construct	2
2.4.1	Looping	2
2.4.2	Conditional	2
2.4.3	Grouping	2
2.5	Functions	2
2.6	Parameters	3
2.7	Expansion	3
2.8	Redirection	3
3	Array	3
4	sed	4
5	awk	4
6	grep	4
7	find	4

1 Introduction

<https://www.gnu.org/software/bash/manual/bash.html>

2 Shell Syntax

2.1 Quoting

Definition 2.1 (quoting). *Quoting is used to remove the special meaning of certain characters or words to the shell.*

```
echo "${var}"
echo "$(date)"
echo "`date`"
echo -e "\t"
# \n : newline
# \t : horizontal tab
# \r : carriage return
```

2.2 Pipelines

Definition 2.2 (pipeline). *cmd1 output* $\rightarrow$ *cmd2 input*

```
cmd1 | cmd2      # stdout
cmd1 |& cmd2     # stderr to stdout
cmd1 2>&1 | cmd2 # stderr to stdout
```

2.3 Lists of Commands

Definition 2.3 (list).

```
cmd1 && cmd2 # And, cmd1 true then cmd2
cmd1 || cmd2 # Or, cmd1 false then cmd2
cmd1; cmd2   # or terminated by newline
cmd1 & cmd2  # cmd1 async
```

2.4 Construct

2.4.1 Looping

Definition 2.4 (looping).

```
for item in item1 item2 ...; do
    cmd
done
```

Example 2.5.

```
for i in ${!array[@]}; do
    echo $array[$i]
done
```

2.4.2 Conditional

Definition 2.6 (conditional).

```
if [[ cond1 ]]; then
    cmd1
elif [[ cond2 ]]; then
    cmd2
else
    cmd3
fi
```

```
# cond:
# ==
# !=
# =~ : unquoted regex
```

2.4.3 Grouping

Definition 2.7 (grouping).

```
( expr ) # subshell env, def var vanish when exit
{ expr; } # ; or newline is required
```

2.5 Functions

Definition 2.8 (function).

```
func () {
    local var = $1
}
```

2.6 Parameters

Definition 2.9 (parameters).

- `${N}` : *position param*
- `$@` : *expands to all pos params, \$1 \$2 ...*
- `$0` : *file or shell name*

2.7 Expansion

Definition 2.10 (brace expansion). *Generate strings sharing a common prefix and suffix.*

```
echo file{1,2} # file1 file2
```

Definition 2.11 (tidle expansion).

```
~ # $HOME
```

Definition 2.12 (parameter expansion).

```
${param}  
array=(0 1 2 3)  
echo ${array[@]} # 0 1 2 3
```

Definition 2.13 (command substitution).

```
$(cmd)  
`cmd`
```

Definition 2.14 (arithmetic expansion).

```
$((expr)) # integer-only
```

Definition 2.15 (filename expansion).

```
* # any string  
? # single char  
[] # char enclosed between []  
*()  
?()  
+()  
!(pattern list) # match except given patterns
```

2.8 Redirection

Definition 2.16 (redirection).

```
ls > file 2>&1 # stdout and stderr to file  
ls &> file # or  
pwd >> file 2>&1 # append stdout and stderr to file  
pwd &>> file # or
```

3 Array

Definition 3.1 (array). `array=(v1 v2 ...)`

4 sed

5 awk

<https://www.gnu.org/software/gawk/manual/gawk.html>

Definition 5.1 (awk). *The basic function of awk is to search files for lines that contain certain patterns.*

Example 5.2.

```
# % @fig{png}
mapfile -t array < <(awk -F'[{}]' '/^% @fig\{/ { print $2 }' file)
```

6 grep

Definition 6.1 (grep). *Search input file for matches to the patterns.*

7 find